

Foreground and Background Threads in C# with Examples

In this article, I am going to discuss **Foreground and Background Threads in C#** with Examples. Please read our previous article where we discussed [Thread Pool in C#](#) with Examples.

Types of Threads in C#

Threading is one of the most important concepts and as a developer, we all need to understand this concept because in most real-time applications we use threading. In C#, by default, the execution of an application starts with a thread which is called the main thread which is automatically run by the CLR and the Operating System.

From the main thread, we can also create other threads for doing the desired tasks simultaneously or concurrently in the application. In C#, we can create two types of threads. They are as follows.

1. **Foreground Thread**
2. **Background Thread**

Foreground Thread in C#:

Foreground threads are those threads that keep running even after the main application exits or quits. That means if the Main thread leaves the application, then still the foreground threads are running to complete their assigned task. So, the foreground threads in C# do not care whether the main thread is alive or not, it completes only when it finishes its assigned work. That means the life of a foreground thread in C# does not depend upon the main thread. The foreground thread has the ability to prevent the current application from terminating. The CLR will not shut down the application until all the Foreground Threads have finished their assigned work. The main thread is a foreground thread.

Background Thread in C#:

Background Threads are those threads that will quit if our main application quits. Or in simple words, we can say that the life of the background thread will depend upon the life of the main thread. In short, if our main application quits, then the background threads will also quit. Background threads are viewed by the CLR and if all foreground threads have terminated, then all the background threads are automatically stopped when the application quits.

Note: By default, when we created a thread in C#, it is a Foreground thread.

How to Make a Thread as Background Thread in C#?

As we already discussed, in C#, the thread is created and managed by the Thread class. The Thread class provides a property called `IsBackground` to check whether the given thread is running in the background or in the foreground. If the value of `IsBackground` property is set to true, then the thread is a background thread i.e. running in the background. Or if the value of `IsBackground` is set to false, then the thread is a foreground thread i.e. running in the foreground.

1. **`IsBackground {get; set;}`**: This property is used to get or set a value indicating whether or not a thread is a background thread. It returns true if this thread is or is to become a background thread; otherwise, false. It throws `ThreadStateException` if the thread is dead.

By default, when we create a thread in C# by using the Thread class, the IsBackground property is false, and if you want to make a background thread in your program, then you need to set the value of IsBackground property of the thread to true.

Example to Understand Foreground Thread in C#:

By default when we created a thread in C#, it is a Foreground Thread. As we discussed Foreground Threads are the threads that keep running even when the main application or main thread quits. Let us understand this with an example.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1);
            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
        }
        // Static method
        static void Method1()
        {
            Console.WriteLine("Method1 Started");
            for (int i = 0; i <= 5; i++)
            {
                Console.WriteLine("Method1 is in Progress!!");
                Thread.Sleep(1000);
            }
            Console.WriteLine("Method1 Exited");
            Console.WriteLine("Press any key to Exit.");
            Console.ReadKey();
        }
    }
}
```

Output:

```
Thread1 is a Background thread: False
Main Thread Exited
Method1 Started
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 is in Progress!!
Method1 Exited
Press any key to Exit.
```

As you can see in the above output, after the main thread or main method completes its execution, still the Foreground Thread is running. So, this proves that Foreground threads keep running and complete their tasks even after the application exits or quits i.e. even after the main thread quits.

Example to Understand Background Thread in C#:

By default when we created a thread in C#, it is a Foreground Thread. And, if you want to make a thread as a background thread, then you need to set the `IsBackground` property of the thread object to true. Background Threads are the threads that will quit if our main thread quits i.e. if all the foreground threads quit, then automatically the background thread quits. Let us understand this with an example. We are rewriting the previous example and here we simply set the `IsBackground` property to true of the thread object.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1)
            {
                //Thread becomes background thread
                IsBackground = true
            };

            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
            //As the Main thread (i.e. foreground thread exits the application
            //Automatically, the background thread quits the application
        }
        // Static method
```

```

static void Method1()
{
    Console.WriteLine("Method1 Started");
    for (int i = 0; i <= 5; i++)
    {
        Console.WriteLine("Method1 is in Progress!!");
        Thread.Sleep(1000);
    }
    Console.WriteLine("Method1 Exited");
    Console.WriteLine("Press any key to Exit.");
    Console.ReadKey();
}
}
}

```

Output:

```

Thread1 is a Background thread: True
Main Thread Exited

```

In the above example, as soon as the main thread quits the background thread also quit.

Multiple Foreground Threads and one Background Thread in C#

In the below example, the main thread is by default a foreground thread and the main thread creates a thread1 object to call Method1. Here, thread1 is also a foreground thread. Then from Method1, we created a foreground thread to call Method2. Here, once all the foreground threads i.e. Main thread and thread1 quit, then automatically the background thread i.e. thread2 quits the application without completing its task (sometimes the task might be completed).

```

using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // A thread created here to run Method1 Parallely
            Thread thread1 = new Thread(Method1)
            {
            };
            Console.WriteLine($"Thread1 is a Background thread:
{thread1.IsBackground}");
            thread1.Start();
            //The control will come here and will exit
            //the main thread or main application
            Console.WriteLine("Main Thread Exited");
            //As the Main thread (i.e. foreground thread exits the applicatio

```

```

        //Automatically, the background thread quits the application
    }
    // Static method
    static void Method1()
    {
        Console.WriteLine("Method1 Started");
        Thread thread2 = new Thread(Method2)
        {
            IsBackground = true
        };
        thread2.Start();
        Thread.Sleep(3000);
        Console.WriteLine("Method1 Exited");
    }
    // Static method
    static void Method2()
    {
        Console.WriteLine("Method2 Started");
        for (int i = 0; i <= 10; i++)
        {
            Console.WriteLine("Method2 is in Progress!!");
            Thread.Sleep(1000);
        }
        Console.WriteLine("Method2 Exited");
        Console.WriteLine("Press any key to Exit.");
        Console.ReadKey();
    }
}

```

So, the point that you need to remember is once all the foreground thread including the main thread quits the application, then automatically all the background threads quit the application.

Points to Remember:

In C#, a thread is either a background thread or a foreground thread. Background threads are similar to the foreground threads, except that the background threads do not prevent a process from terminating. Once all the Foreground threads belonging to a process have terminated, then the CLR ends the process. Any remaining background threads are stopped and not completed.

By default, the following threads execute in the foreground (that is, their `IsBackground` property returns false):

1. The primary thread (or main application thread).
2. All threads are created by calling a `Thread` class constructor.

By default, the following threads execute in the background (that is, their `IsBackground` property returns true):

1. Thread pool threads are a pool of worker threads maintained by the runtime. You can configure the thread pool and schedule work on thread pool threads by using the `ThreadPool` class.

Example for Better understanding Background and Foreground Threads in C#:

In the below example, we are showing the behavior of foreground and background threads in C#. The code example will create a foreground thread and a background thread. The foreground thread keeps the process running until completes its for loop and terminates. The foreground thread has finished execution, the process is terminated before the background thread has completed execution.

```
using System;
using System.Threading;
namespace MultithreadingDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            ThreadingTest foregroundTest = new ThreadingTest(5);
            //Creating a Foreground Thread
            Thread foregroundThread = new Thread(new
ThreadStart(foregroundTest.RunLoop));
            ThreadingTest backgroundTest = new ThreadingTest(50);
            //Creating a Background Thread
            Thread backgroundThread = new Thread(new ThreadStart(backgroundTest
            {
                IsBackground = true
            }
            );
            foregroundThread.Start();
            backgroundThread.Start();
        }
    }
    class ThreadingTest
    {
        readonly int maxIterations;
        public ThreadingTest(int maxIterations)
        {
            this.maxIterations = maxIterations;
        }
        public void RunLoop()
        {
            for (int i = 0; i < maxIterations; i++)
            {
                Console.WriteLine($"{0} count: {1}", Thread.CurrentThread.IsBack
"Background Thread" : "Foreground Thread", i);
                Thread.Sleep(250);
            }
            Console.WriteLine("{0} finished counting.", Thread.CurrentThread.I
? "Background Thread" : "Foreground Thread");
        }
    }
}
```

Note1: In C#, the foreground threads have the ability to prevent the current application from terminating. The CLR will not shut down an application (which is to say, unload the hosting AppDomain) until all foreground threads have ended.

Note2: In C#, the Background threads are viewed by the CLR as expendable paths of execution that can be ignored at any point in time (even if they are currently doing some unit of work). Thus, if all foreground threads have terminated, any and all background threads are automatically killed when the application domain unloads.

In the next article, I am going to discuss [AutoResetEvent and ManualResetEvent in C#](#) with Examples. Here, in this article, I try to explain **Foreground and Background Threads in C#** with Examples. I hope you enjoy this Foreground and Background Threads in C# article.